

Operaciones Prácticas con List<T> e IList<T>

Tecnicatura Superior en Desarrollo de Software

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat |
Ciclo Lectivo 2026

¿Qué es?

Esta sección documenta las operaciones fundamentales de manipulación de colecciones genéricas en C#. Se analiza cómo agregar, consultar, modificar, reordenar y transformar elementos dentro de una estructura dinámica. Cada operación expone un contrato claro a través de `IList<T>`, mientras que la implementación concreta `List<T>` gestiona la memoria interna de forma transparente.

Analogía

Imagine una pizarra magnética con ranuras numeradas. Agregar un imán nuevo al final es rápido y no requiere mover los existentes. Insertar un imán en el medio obliga a desplazar todos los siguientes hacia la derecha. Quitar uno de la mitad genera un espacio vacío que se cierra automáticamente, desplazando los restantes hacia la izquierda. Buscar un imán específico requiere revisarlos uno por uno hasta encontrar coincidencia. Cada operación tiene un costo físico distinto, pero la pizarra se encarga de mantener el orden sin intervención manual.

Características

- **Mutabilidad controlada:** Permite modificar la colección después de su creación sin recrearla.
- **Acceso indexado O(1):** Obtención o actualización instantánea por posición conocida.

- **Búsqueda lineal:** Métodos como `Contains` recorren elementos secuencialmente hasta hallar coincidencia.
- **Reordenamiento in-place:** `Sort()` modifica la colección original sin generar copias.
- **Transformación funcional:** `ConvertAll()` proyecta cada elemento a un nuevo tipo o valor, retornando una lista independiente.

¿Por qué se usa?

Las colecciones estáticas (arreglos simples) obligan a definir tamaño fijo en compilación y no permiten inserciones o eliminaciones sin copiar datos manualmente. `List<T>` resuelve esto gestionando un arreglo interno redimensionable. Al exponer la colección mediante `ICollection<T>`, el código consumidor depende únicamente del contrato, facilitando pruebas unitarias, cambios de implementación futura y mantenimiento a largo plazo.

Cada método expuesto responde a un patrón de uso real: carga incremental, validación de rangos, eliminación segura, ordenamiento para presentación y transformación para cálculo. Comprender el *por qué* detrás de cada llamada evita errores silenciosos como divisiones enteras truncadas, excepciones de índice fuera de rango o modificación de colecciones durante iteración.

Arquitectura



Cada operación tiene un costo algorítmico distinto. `Add()` es $O(1)$ amortizado. `Insert()` y `Remove()` son $O(n)$ por el desplazamiento de memoria. `Sort()` es $O(n \log n)$ y modifica la colección original. `ConvertAll()` es $O(n)$ pero retorna una instancia separada, preservando la original.

Entendiendo el "costo" de las operaciones

Antes de analizar cada método, es fundamental comprender qué significa "costo algorítmico". No se refiere a dinero, sino a **esfuerzo, tiempo o pasos necesarios** para completar una tarea. En programación, medimos este esfuerzo en función de cuántos elementos tiene la colección: si la lista tiene 10 elementos o 10.000, ¿cuánto más trabajo requiere la operación? Esta métrica se expresa con la notación $O(\dots)$, que describe cómo crece el esfuerzo a medida que crecen los datos.

Notación Big O simplificada:

- $O(1)$ = "Costo constante": siempre toma el mismo esfuerzo, sin importar el tamaño.
- $O(n)$ = "Costo lineal": el esfuerzo crece proporcionalmente con la cantidad de elementos.
- $O(n \log n)$ = "Costo logarítmico-lineal": crece más rápido que lineal, pero mucho más lento que cuadrático.
- "Amortizado" = "En promedio": algunas operaciones son rápidas la mayoría de las veces, pero ocasionalmente requieren un esfuerzo extra que se "promedia" en el tiempo.

Analogía central: La biblioteca escolar

Imagine una biblioteca con estanterías numeradas. Cada libro representa un elemento de la colección. El bibliotecario (el programa) debe realizar tareas como agregar libros nuevos, reubicar ejemplares, ordenar por autor o crear catálogos temáticos. Cada tarea tiene un esfuerzo distinto según cómo esté organizada la biblioteca y cuántos libros haya.

Add() — $O(1)$ amortizado

Analogía: Agregar un libro nuevo al *final* de una estantería que tiene espacio reservado. El bibliotecario simplemente coloca el libro en el siguiente casillero libre. No necesita mover ningún otro libro. El esfuerzo es siempre el mismo: caminar hasta el final y depositar.

¿**Por qué "amortizado"**? Imaginemos que la estantería tiene 20 casilleros y ya está llena. Para agregar el libro número 21, el bibliotecario debe: (1) conseguir una estantería nueva de 40 casilleros, (2) copiar los 20 libros existentes a la nueva, (3) colocar el libro nuevo en la posición 21. Este evento es costoso, pero ocurre muy pocas veces. La mayoría de las veces, simplemente se agrega al espacio libre. El "costo promedio" por operación sigue siendo bajo.

Ejemplo cotidiano:

Añadir una hoja al final de un bloc de notas con espiral.

- Si hay espacio: girar la espiral y colocar → esfuerzo mínimo.
- Si el bloc está lleno: comprar uno nuevo, copiar todas las hojas, añadir la nueva → esfuerzo alto, pero ocurre rara vez.

El esfuerzo "promedio" por hoja añadida sigue siendo bajo.

Insert(index, T) y Remove(T) — $O(n)$

Analogía para Insert: Querer colocar un libro en la posición 5 de una estantería ya ocupada. El bibliotecario debe: (1) mover el libro que está en la posición 5 a la 6, (2) el de la 6 a la 7, y así sucesivamente hasta el final, para "hacer espacio". Luego coloca el nuevo libro en la posición 5. Cuantos más libros haya después de la posición de inserción, más movimientos requiere.

Analogía para Remove: Retirar un libro de la posición 5. Al quitarlo, queda un hueco. Para mantener la estantería compacta (sin espacios vacíos intermedios), el bibliotecario debe: (1) mover el libro de la posición 6 a la 5, (2) el de la 7 a la 6, y así hasta el final. Nuevamente, cuantos más elementos haya después del punto de eliminación, más trabajo implica.

```
Visualización del desplazamiento:  
Posición: [1][2][3][4][5][6][7][8]  
Eliminar en [4]:  
→ Quitar elemento 4 → hueco  
→ Mover 5→4, 6→5, 7→6, 8→7  
→ Resultado: [1][2][3][5][6][7][8][ ]  
Cada flecha representa un movimiento físico.  
Si hay n elementos después del punto, se requieren n movimientos.
```

Sort() — $O(n \log n)$, modifica la colección original

Analogía: Ordenar todos los libros de la biblioteca por apellido de autor. El bibliotecario no puede simplemente "colocar" cada libro en su lugar definitivo de una vez, porque al mover uno, puede desordenar otros ya ubicados. Debe comparar pares de libros, intercambiarlos si están fuera de orden, y repetir este proceso hasta que toda la colección esté organizada. Los algoritmos eficientes (como el que usa .NET) dividen la colección en grupos pequeños, los ordenan, y luego los fusionan. Este enfoque reduce drásticamente las comparaciones necesarias, pero aún requiere revisar múltiples veces los elementos.

Modifica la original: Al terminar, los libros ya no están en el orden en que fueron ingresados. La estantería física cambió. No hay una "copia" preservada; el estado anterior se perdió.

Comparación con la vida real:

Ordenar una baraja de cartas mezcladas.

- No puedes saber de antemano dónde va cada carta.
- Debes comparar cartas entre sí, intercambiar posiciones, repetir.
- Al final, la baraja original ya no existe: fue transformada.
- El esfuerzo no es proporcional al número de cartas, sino que crece un poco más rápido.

ConvertAll() — $O(n)$, retorna una instancia separada

Analogía: Crear un catálogo temático a partir de los libros existentes. Por ejemplo, tomar cada libro de la estantería principal y escribir en una ficha nueva solo el título y el año, ignorando el autor y la editorial. El bibliotecario recorre la

estantería una vez (n elementos), y por cada libro, realiza una tarea fija: copiar dos datos a una ficha nueva. Al finalizar, tiene un conjunto nuevo de fichas, pero la estantería original permanece intacta, con todos sus libros en su lugar.

Preserva la original: Esta operación es "no destructiva". La colección fuente no se modifica; se genera una nueva colección con los resultados transformados. Esto permite mantener el estado original para otras operaciones o para auditoría.

Resumen visual de costos:

Operación	Costo	Efecto
Add()	$O(1)^*$	Modifica
Insert()	$O(n)$	Modifica
Remove()	$O(n)$	Modifica
Sort()	$O(n \log n)$	Modifica
ConvertAll()	$O(n)$	Nueva instancia

* Amortizado: costo promedio bajo, con picos ocasionales.

* Amortizado: costo promedio bajo, con picos ocasionales.

¿Por qué importa esto para el desarrollador?

Conocer el costo de cada operación permite tomar decisiones informadas al diseñar software. Por ejemplo:

- Si se van a agregar muchos elementos uno por uno, `Add()` es eficiente. Pero si se conoce de antemano la cantidad aproximada, inicializar la lista con una capacidad estimada evita los costosos redimensionamientos intermedios.
- Si se necesitan inserciones frecuentes en posiciones arbitrarias, una `List<T>` puede no ser la estructura óptima; otras colecciones (como `LinkedList<T>`) tienen costos distintos para esos escenarios.
- Si se requiere ordenar pero también preservar el orden original, se debe copiar la lista antes de llamar a `Sort()`, o usar `OrderBy()` que retorna una nueva secuencia.
- Si se transforma datos para presentación (ej: convertir entidades a DTOs), `ConvertAll()` ofrece claridad y seguridad al no alterar la fuente.

Advertencia pedagógica:

En esta etapa del curso, el objetivo es *comprender* estos conceptos, no memorizar fórmulas. La notación Big O se profundizará en asignaturas posteriores de Algoritmos. Por ahora, basta con recordar: "algunas operaciones son rápidas siempre, otras se vuelven más lentas cuantos más datos haya, y algunas cambian la colección mientras que otras dejan la original intacta".

Marco teórico

El manejo de colecciones dinámicas requiere comprender no solo qué método invocar, sino por qué la arquitectura de .NET lo diseñó de esa manera. A continuación, se presentan diez escenarios prácticos. Cada caso expone primero la justificación técnica y luego la implementación concreta, con comentarios que detallan el comportamiento interno.

1. Carga incremental y contrato `ICollection<T>`

Declarar la variable con `ICollection<T>` permite que el consumidor interactúe únicamente con el contrato. La instanciación con `new List<T>()` reserva un arreglo interno de capacidad cero, que se expande automáticamente al primer elemento. El método `Add()` se utiliza porque encapsula la verificación de capacidad y el redimensionamiento transparente. El bucle `foreach` se prefiere sobre índices manuales porque elimina riesgos de desbordamiento y delega la iteración al enumerador interno de la colección.

```
1  using System;
2  using System.Collections.Generic;
3  class Program
4  {
5      static void Main()
6      {
7          ICollection<string> subjects = new List<string>();
8          string input;
9          Console.WriteLine("Ingrese temas (escriba 'fin' para terminar):");
10         do
11         {
12             input = Console.ReadLine();
13             if (input != "fin")
```

```
14         subjects.Add(input);
15     } while (input != "fin");
16     Console.WriteLine("\nTemas cargados:");
17     foreach (var topic in subjects)
18         Console.WriteLine(topic);
19     }
20 }
```

2. Acceso por índice y validación de límites

El acceso directo mediante corchetes `lista[index]` es la operación más rápida, pero lanza `ArgumentOutOfRangeException` si el índice es inválido. La validación explícita con `index >= 0 && index < lista.Count` es obligatoria cuando el índice proviene de entrada externa. Esta verificación previene fallos en tiempo de ejecución y garantiza que la memoria interna solo se accede dentro de sus límites seguros.

```
1  using System;
2  using System.Collections.Generic;
3  class Program
4  {
5      static void Main()
6      {
7          IList<int> grades = new List<int>() { 6, 7, 8 };
8          Console.Write("Ingrese índice a modificar (0-2): ");
9          int index = int.Parse(Console.ReadLine());
10         if (index >= 0 && index < grades.Count)
11         {
12             Console.Write("Nueva nota: ");
13             grades[index] = int.Parse(Console.ReadLine());
14         }
15         Console.WriteLine("Notas:");
16         foreach (var grade in grades)
17             Console.WriteLine(grade);
18     }
19 }
```

3. Búsqueda y eliminación segura

`Contains()` realiza una búsqueda lineal utilizando el método `Equals()` de cada elemento. Para tipos primitivos como `string`, compara el valor exacto. Llamar a `Remove()` directamente sobre un elemento inexistente no lanza excepción, pero retorna `false`. Validar con `Contains()` primero hace explícita la intención y permite manejar casos de "no encontrado" de forma controlada.

```
1 using System;
2 using System.Collections.Generic;
3 class Program
4 {
5     static void Main()
6     {
7         IList<string> products = new List<string>()
8         {
9             "Pan", "Leche", "Azúcar"
10        };
11        Console.Write("Producto a eliminar: ");
12        string target = Console.ReadLine();
13        if (products.Contains(target))
14            products.Remove(target);
15        Console.WriteLine("Lista actual:");
16        foreach (var item in products)
17            Console.WriteLine(item);
18    }
19 }
```

Nota sobre Remove:

El método Remove(T) retorna true si elimina un elemento, false si no lo encuentra.

Nunca se debe ignorar este valor de retorno cuando la lógica posterior depende de la eliminación.

La verificación previa con Contains() evita recorrer la colección dos veces.

4. Inserción controlada y desplazamiento

`Add()` siempre coloca elementos al final. `Insert(index, value)` permite ubicar un elemento en una posición específica, pero requiere desplazar todos los elementos posteriores hacia la derecha. La validación `pos <= list.Count` es crucial: permite insertar al final (índice igual a Count), pero rechaza índices mayores. Esta operación es $O(n)$ y debe evitarse en bucles frecuentes con colecciones grandes.

```
1 using System;
2 using System.Collections.Generic;
3 class Program
4 {
5     static void Main()
6     {
7         IList<string> students = new List<string>()
8         {
```

```
9         "Ana", "Luis", "Pedro"
10     };
11     Console.WriteLine("Nombre del nuevo alumno: ");
12     string name = Console.ReadLine();
13     Console.WriteLine("Posición (0 a " + students.Count + "): ");
14     int position = int.Parse(Console.ReadLine());
15     if (position >= 0 && position <= students.Count)
16         students.Insert(position, name);
17     Console.WriteLine("Lista final:");
18     foreach (var student in students)
19         Console.WriteLine(student);
20     }
21 }
```

5. Operación compuesta y lectura-escritura

La expresión `lista[i] -= valor` combina lectura, cálculo y escritura en una sola línea. Internamente, el runtime obtiene el valor actual, aplica la resta y lo vuelve a almacenar en la misma posición. Validar el rango antes de operar es obligatorio, ya que el operador compuesto no verifica límites automáticamente. Esta técnica es ideal para actualizar contadores o inventarios sin crear variables intermedias.

```
1 using System;
2 using System.Collections.Generic;
3 class Program
4 {
5     static void Main()
6     {
7         IList<int> stock = new List<int>() { 10, 15, 20 };
8         Console.WriteLine("Ingreso índice del producto: ");
9         int index = int.Parse(Console.ReadLine());
10        if (index >= 0 && index < stock.Count)
11        {
12            Console.WriteLine("Cantidad vendida: ");
13            int sold = int.Parse(Console.ReadLine());
14            stock[index] -= sold;
15        }
16        Console.WriteLine("Stock actualizado:");
17        foreach (var item in stock)
18            Console.WriteLine(item);
19    }
20 }
```

6. Ordenamiento in-place y Comparador por defecto

`Sort()` modifica la colección original aplicando un algoritmo de ordenamiento eficiente. Para tipos numéricos y de texto, utiliza la comparación natural ascendente. No requiere parámetros adicionales cuando los elementos implementan `IComparable` (como `int` o `string`). Es fundamental recordar que esta operación altera el estado interno; si se necesita preservar el orden original, debe copiarse la lista antes de ordenar.

```
1  using System;
2  using System.Collections.Generic;
3  class Program
4  {
5      static void Main()
6      {
7          List<int> numbers = new List<int>();
8          for (int i = 0; i < 5; i++)
9          {
10             Console.Write("Número: ");
11             numbers.Add(int.Parse(Console.ReadLine()));
12         }
13         numbers.Sort();
14         Console.WriteLine("Ordenados:");
15         foreach (var n in numbers)
16             Console.WriteLine(n);
17     }
18 }
```

Cálculo de promedios con tipos enteros:

`suma / lista.Count` realiza división entera. El resultado trunca decimales. Para obtener precisión decimal, castear uno de los operandos a `double` o `decimal`.

Ejemplo correcto: `(double)suma / lista.Count`

7. Búsqueda booleana y conteo eficiente

`Contains()` retorna un valor booleano útil para validaciones condicionales. La propiedad `Count` es $O(1)$, ya que almacena internamente la cantidad de elementos y la actualiza en cada mutación. Iterar manualmente para contar elementos es redundante y degrada el rendimiento. Combinar ambas herramientas permite validar existencia y procesar tamaños sin overhead adicional.

```
1 using System;
2 using System.Collections.Generic;
3 class Program
4 {
5     static void Main()
6     {
7         List<string> names = new List<string>();
8         for (int i = 0; i < 3; i++)
9         {
10             Console.Write("Nombre: ");
11             names.Add(Console.ReadLine());
12         }
13         Console.Write("Buscar nombre: ");
14         string search = Console.ReadLine();
15         if (names.Contains(search))
16             Console.WriteLine("Existe");
17         else
18             Console.WriteLine("No existe");
19     }
20 }
```

8. Acumulación y propiedad Count

Este patrón combina la gestión de colecciones con operaciones aritméticas acumulativas. Se utiliza un bucle `for` con límite fijo para solicitar datos. En cada iteración, el valor ingresado se registra en la lista mediante `Add()` y se suma a una variable externa. La propiedad `Count` se emplea para conocer la cantidad exacta de elementos, evitando hardcodear números mágicos en la salida. Es fundamental notar que `(suma / lista.Count)` realiza una división entera, lo que trunca decimales. Si se requiere precisión, debe aplicarse un cast explícito a `double` o `decimal`. La lista se mantiene para preservar los valores individuales, permitiendo futuras validaciones, ordenamientos o filtros sin volver a solicitar datos.

```
1 using System;
2 using System.Collections.Generic;
3 class Program
4 {
5     static void Main()
6     {
7         List<int> grades = new List<int>();
8         int sum = 0;
9         for (int i = 0; i < 4; i++)
10        {
11            Console.Write("Nota: ");
12            int grade = int.Parse(Console.ReadLine());
```

```
13         grades.Add(grade); // Registra el valor individual
14         sum += grade; // Acumula para el cálculo final
15     }
16     Console.WriteLine("Cantidad: " + grades.Count);
17     // División entera: trunca decimales automáticamente
18     Console.WriteLine("Promedio: " + (sum / grades.Count));
19 }
20 }
```

9. Vaciado seguro vs reasignación

`Clear()` elimina todos los elementos pero conserva la capacidad interna del arreglo. Esto permite reutilizar la colección sin realocar memoria. Reasignar `lista = new List<T>()` crea una instancia completamente nueva y deja la anterior pendiente para el recolector de basura. `Clear()` es preferible cuando se recicla el mismo objeto en bucles o procesos repetitivos.

```
1  using System;
2  using System.Collections.Generic;
3  class Program
4  {
5      static void Main()
6      {
7          List<string> data = new List<string>() { "A", "B", "C" };
8          Console.Write("¿Vaciar lista? (s/n): ");
9          string response = Console.ReadLine();
10         if (response == "s")
11             data.Clear();
12         Console.WriteLine("Cantidad final: " + data.Count);
13     }
14 }
```

10. Transformación con ConvertAll

`ConvertAll()` aplica una regla de transformación a cada elemento y retorna una lista nueva con los resultados. El parámetro espera un delegado o expresión lambda. En este caso, `x => x * 2` define una función anónima que recibe un entero y retorna su doble. La lista original permanece intacta. Este patrón es la base de la programación funcional en colecciones y evita bucles `for` manuales para mapeo de datos.

```
1  using System;
2  using System.Collections.Generic;
3  class Program
4  {
```

```
5 static void Main()
6 {
7     List<int> values = new List<int>();
8     int count;
9     Console.Write("¿Cuántos números ingresar?: ");
10    count = int.Parse(Console.ReadLine());
11    for (int i = 0; i < count; i++)
12    {
13        Console.Write("Número: ");
14        values.Add(int.Parse(Console.ReadLine()));
15    }
16    List<int> doubles = values.ConvertAll(x => x * 2);
17    Console.WriteLine("Duplicados:");
18    foreach (var n in doubles)
19        Console.WriteLine(n);
20 }
21 }
```

Errores comunes

ERROR COMUN

Acceder a un índice sin validar, generando

`ArgumentOutOfRangeException` en tiempo de ejecución.

```
1 IList<int> data = new List<int>() {
2     int val = data[userInput]; // Si us
```

La ausencia de validación de límites confía ciegamente en la entrada externa. Cualquier índice fuera de `[0, Count-1]` lanza excepción no controlada.

BUENA PRACTICA

Validar explícitamente el rango antes de acceder o modificar por índice.

```
1 IList<int> data = new List<int>() {
2     if (userInput >= 0 && userInput < d
3     {
4         int val = data[userInput]; // A
5     }
```

La verificación previa garantiza que la operación solo ocurre dentro de límites válidos. El flujo continúa sin interrupciones incluso con entrada errónea.

Qué NO hacer

QUE NO HACER

Modificar la colección dentro de un bucle `foreach`, lo que invalida el enumerador interno y lanza `InvalidOperationException`.

```
1 List<int> items = new List<int>() {  
2     foreach (var item in items)  
3     {  
4         if (item % 2 == 0)  
5             items.Remove(item); // Lanz  
6     }
```

`foreach` mantiene un puntero interno al estado actual de la colección. Cualquier mutación rompe la consistencia del enumerador.

BUENA PRACTICA

Iterar hacia atrás con `for` o usar `RemoveAll()` para filtrar sin invalidar la iteración.

```
1 List<int> items = new List<int>() {  
2     for (int i = items.Count - 1; i >=  
3     {  
4         if (items[i] % 2 == 0)  
5             items.RemoveAt(i); // Itera  
6     }
```

Recorrer desde el final hacia el inicio mantiene válidos los índices de los elementos no procesados. `RemoveAll()` también es una alternativa declarativa segura.

Modificar colecciones durante iteración es un error estructural que rompe el contrato del enumerador. Siempre se debe preferir iteración inversa con `for`, o métodos de filtrado interno que manejan el desplazamiento de memoria de forma atómica. Esto garantiza estabilidad en producción y evita excepciones silenciosas en lógica condicional compleja.